

Document number: P2182R0
Date: 2020-06-15
Audience: SG21
Reply-to: *Andrzej Krzemiński <akrzemi1 at gmail dot com>*
Ryan McDougall
Joshua Berne

Contract Support: Defining the Minimum Viable Feature Set

We observe that "C++20 contracts" consisted of two parts. One part was clear, well understood, widely agreed upon, and useful to various groups of C++ users. The other part rose a lot of controversies for various reasons and ultimately led to the removal of the entire feature.

In paper [\[P2114r0\]](#) we have listed a subset of use cases we believe represent useful improvements to the language, offer a incalculable positive value for most users, and reflect the uncontroversial part of "C++20 contracts". In this paper we provide the rationale for our choice. We wish to show that our selection:

- is big enough to be called a Minimum Viable Product (MVP) and get support of the majority of C++ programmers
- does not require controversial design, or features not already implemented in other languages
- does not prevent the addition of any of the remaining features of "C++20 contracts", or other discussed alternatives, in subsequent stages

Analysis

In this document we use the term *C++20 Contracts* to refer to contract support facilities in C++ Standard draft just before the removal — [\[N4820\]](#) (which is effectively equivalent to [\[P0542R5\]](#)) — as well as pieces of the associated proposals seen by EWG that had support in polling, such as:

- [\[P1290R3\]](#)
- [\[P1344R1\]](#)
- [\[P1429R3\]](#)
- [\[P1607R1\]](#)
- [\[P1711R0\]](#)

[\[P2076r0\]](#) lists the points the disagreements about the previous incarnation of contracts design:

- Continuation Mode
- Build Levels
- Global Toggles (also known as: external remapping controls)
- Literal Semantics (also known as: in-source controls)
- Assumptions

We claim that if we eliminated or reduced the use cases for the above features, the remainder would still be a valuable and usable feature that would offer a great help in enforcing program correctness. The value in preventing bugs and enforcing program correctness comes from being able to

1. express preconditions, postconditions, and assertions as C++ predicates
2. transform them into runtime-checks at an all-or-none granularity, leaving open the design of finer control to future work

In the reminder of the paper we elaborate on the features of MVP and show how they are motivated by the use cases nominated in [\[P2114r0\]](#). We also discuss how the support for the remaining use cases can be added atop of MVP in subsequent phases.

Features in MVP

We selected the minimal use cases to allow the widest benefit for the most general section of users, without resorting to known points of contention. We had in mind a simple feature that consisted of the existing contract syntax, a single build option to enable or disable checks, and a (conditionally) user- definable violation handler that is invoked when checks are enabled and violated.

Contract declarations

The most important thing is the ability to express the predicates in function declarations as compiler-checked C++ expressions. This is uncontroversial. This enables:

- compiler to report type-system errors from incorrectly spelled constraints
- generation of run-time code — such as contract checks, or unit tests
- annotation aware static analysis

In fact, this feature alone would already satisfy many users.

Use cases covered:

- [dev.reason.knowl](#)
- [dev.readable.syntax](#)
- [dev.readable.keywords](#)
- [dev.parsable](#)
- [dev.tooling](#)
- [cppdev.syntax.familiar](#)
- [cppdev.syntax.cpp](#)
- [cppdev.syntax.reuse](#)
- [cppdev.location](#)
- [api.communicate.inputsoutputs](#)
- [api.establish.preconditions](#)
- [api.establish.postconditions](#)
- [api.express.values](#)
- [api.contract.errorhandling](#)
- [arch.nomacros](#)
- [arch.complete](#)
- [sdev.bestpractices](#)
- [jdev.understand.contracts](#)
- [jdev.understand.keywords](#)
- [adev.fast](#)
- [teach.bestpractices](#)
- [teach.standardized](#)
- [teach.portable](#)

Turning on and off all runtime checks

We believe it is essential that we meet two somewhat contradictory requirements:

1. A guarantee that some action is taken whenever a contract is violated
2. A guarantee that there is no runtime overhead for encoded contracts

The former is a correctness requirement, and the latter is performance requirement (after assurance — further discussion about assurance can be found in [\[P1517R0\]](#)).

This suggests that there needs to be at least some control over what contracts can do at build time. For the MVP we suggest a single flag which turns checking "on" or "off", where:

- Off is simply syntax-checked
- On evaluates the predicate and invokes a violation handler when the predicate is false

This is a subset of what [\[P0542R5\]](#) offered, and should come with similar caveats that that proposal had for build modes — where mixing modes is only conditionally supported.

Finer-grained control and additional semantics to failed checks would be left to extensions and future proposals.

Use cases covered:

- [dev.reason.sideeffects](#)
- [int.conform.postconditions](#)
- [int.control.build](#)
- [int.runtime.unchecked](#)
- [qdev.checkall](#)

Reporting contract violation

On platforms where it makes sense, each contract violation detected at run-time should be reported to the users — including what failed and where in the source code.

Use cases covered:

- [api.establish.check](#)
- [api.establish.values](#)
- [int.conform.violation](#)
- [int.violations.information](#)
- [jdev.understand.violations](#)
- [jdev.understand.aborting](#)
- [jdev.understand.buildviolation](#)

Customizing how failures are reported

The semantics and implementation of contract violation handling has significant impact on the extensibility and usability of the feature. It may be premature to codify within an MVP, as it may reduce the applicability of the feature to a significant portion of use cases. For the MVP, we think that as offered in [\[P0542R5\]](#), implementations are not required to offer customizable violation handlers.

Note that in practice, it is expected that most implementations will allow users to define a custom violation handler. This will let users experiment with many variations before bringing them to the committee, and do so in a way that is portable across all compilers that do allow customizability. Compilers that do not allow user-defined violation handlers would instead document what their (default or other selectable) violation handler will do when invoked.

Other use cases related to contract semantics can be met or at least approximated with the choice of specific violation handlers, and we seek to avoid limiting that decision in this proposal. For this reason we would not propose requiring that user defined violation handlers be `[[noreturn]]`, and we suggest no other behavior related to a violation other than the invocation of the violation handler

Use cases covered (implementation defined):

- [int.violations.common](#)

Features not in MVP

Continuation after failed run-time check

The use case that continuation mode was trying to address are [sdev.maturity](#) and [large.newcompiler](#). They can be introduced on top of MVP as annotations on individual contract declarations (as described in [\[P1332R0\]](#)), and additionally requiring more build modes.

In addition to that, by only specifying that a violation handler is invoked and not extending that to any other fixed functionality associated with a violation (such as always terminating) we allow for users to remain in control of this decision when choosing compilers that provide selectable violation handlers.

Contract Levels

The use cases that contract levels, such as `default` and `audit`, tried to address are [crit.more.coverage](#), [dev.reason.cost](#) and [crit.production.checking](#). The goal was to indicate "this predicate is more expensive to evaluate than the function it protects", but the design of this feature had flaws:

- Syntax cannot handle templates where the level may depend on template arguments.
- Two levels (or even three) cannot be enough.
- Levels is not enough to convey essential meta information.
- Syntax is not extensible.
- Syntax is uncomfortable.

They can be introduced on top of MVP as annotations indicating how expensive a contract statement is relative to the body of the function.

Global Toggles (also known as: external remapping controls)

If there are no levels in contract declarations in the MVP, there is no need for having build levels either, other than on/off switch. They can be added later if needed.

Literal Semantics (also known as: in-source controls)

They were never in the draft, but were discussed widely, and had some EWG support in polling. There were concerns that they

1. would encourage the use of macros
2. were not basic/necessary building blocks

However they can easily be added atop of MVP in future version by adding meta-information to contract declarations.

Assumptions or injected facts

This was highly controversial. An extended analysis can be found in [P2064r0]. Use cases covered: [pdev.footgun](#) and [hardware.performance](#).

[pdev.footgun](#) can be handled by a separate feature, such as [P1774r3]. Alternatively, it could be implemented atop of MVP by adding special meta-information to individual contract declarations.

[hardware.performance](#) could be implemented atop of MVP by introducing a yet another build mode. Alternatively, users could achieve it with MVP by installing a custom violation handler with `__builtin_unreachable()`.

Indicating inexpressible predicates

This would cover use cases [gdev.tooling.control](#) and [api.express.unimplementable](#). Controversies arose because of mixing inexpressibility with levels, and taking word "axiom" to mean `__builtin_assume()`.

Can be implemented atop of MVP either as meta-information on contract declarations, or by annotating functions as inexpressible as described in [P2176r0].

Conclusion

While the proposed MVP is conservative, we believe that even the MVP may contain a number of unresolved small issues that will require time to polish. With EWG attention almost entirely consumed by controversial bits of C++20 contracts, the uncontroversial bits may not have been given sufficient consideration for attention. If we are to ship any contract programming support in C++23 time frame, we have to be sure that we have a small enough scope to be accommodated by the WG21 process.

The biggest advantage of this MVP is that we provide a standard notation for expressing preconditions and postconditions. This notation will be consumed by many tools, which are not necessarily compilers. The positive effect on the community is not necessarily in what compilers can do with it, but also what other automated tools can get from it. This is far beyond the scope of the C++ Standard.

References

- [P2114r0] — Joshua Berne, Ryan McDougall, Andrzej Krzemiński, "Minimal Contract Use Cases" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2114r0.html>).
- [P1995r0] — Joshua Berne, Timur Doumler, Andrzej Krzemiński, Ryan McDougall, Herb Sutter, "Contracts — Use Cases" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1995r0.html>).
- [P2185r0] — Caleb Sunstrum, "Contracts Use Case Categorization" (<https://isocpp.org/files/papers/P2185R0.html>).
- [N4820] — Richard Smith, "Working Draft, Standard for Programming Language C++", <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf>.
- [P0542r5] — G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, "Support for contract based programming in C++" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0542r5.html>).
- [p2076r0] — Ville Voutilainen, "Previous disagreements on Contracts" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2076r0.html>).
- [P1774r3] — Timur Doumler, "Portable assumptions" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1774r3.pdf>).

- [P1332r0] — Joshua Berne, Nathan Burgers, Hyman Rosen, John Lakos, "Contract Checking in C++: A (long-term) Road Map"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p1332r0.txt>).
- [P1517r0] — Ryan McDougall, "Contract Requirements for Iterative High-Assurance Systems"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1517r0.html>).
- [P2176r0] — Andrzej Krzemiński, "A different take on inexpressible conditions"
(<https://isocpp.org/files/papers/P2176R0.html>).
- [P2064r0] — Herb Sutter, "Assumptions"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2064r0.pdf>).
- [P2038r0] — Andrzej Krzemiński, Ryan McDougall, "Proposed nomenclature for contract-related proposals"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p2038r0.html>).
- [N4160] — Andrzej Krzemiński, "Value Constraints"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4160.html>).
- [P1344r1] — Nathan Myers, "Pre/Post vs. Enspects/Exsures"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1344r1.md>).
- [P1711r0] — Bjarne Stroustrup, "What to do about contracts?"
(<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1711r0.pdf>).